

Reinforcement Learning for Autonomous Rover Resource Extraction: Comparing Tabular Q-Learning and Deep Q-Networks

Authors: James Lambert and Jacob Nelson
University of Colorado Boulder

Abstract—We address the autonomous rover resource extraction problem, where a lunar rover must extract and deposit mineral resources while managing battery and mass constraints. We formulate this as a Markov Decision Process and compare two reinforcement learning approaches: tabular Q-learning operating on discretized states, and Deep Q-Networks (DQN) operating on continuous states with raw field observations. Using physically motivated cost parameters derived from lunar operations economics, we train both agents on a simulated $1000\text{ m} \times 1000\text{ m}$ field with spatial resource distribution. We present learning curves, final policy performance metrics, spatial behavior analysis with trajectory visualizations, and a comparative discussion of sample efficiency, convergence behavior, and computational trade-offs. Our results demonstrate that DQN achieves $4\times$ higher net returns and superior sample efficiency while tabular Q-learning offers faster convergence and simpler deployment characteristics.

I. INTRODUCTION

Autonomous rovers for space exploration must operate under strict resource constraints: limited battery capacity, payload mass restrictions, and complex mission objectives. The ability to make intelligent decisions about where to traverse, when to extract resources, and when to return to the depot is critical for mission success.

Reinforcement learning offers a promising approach to learn optimal extraction policies directly from task performance feedback, without explicit manual programming of behaviors. However, the choice of representation (discrete vs. continuous states) and the learning algorithm significantly impact both computational efficiency and policy quality.

In this project, we develop and compare two complementary approaches:

- **Tabular Q-Learning:** A classical approach that discretizes the continuous rover state into grid bins, enabling a finite tabular representation of the action-value function.
- **Deep Q-Networks (DQN):** A deep reinforcement learning approach that learns an approximation of the state-action value function using a neural network, allowing direct learning from continuous observations.

This report presents our problem formulation, solution approaches, experimental results, and comparative analysis. Our goal is to understand the trade-offs between these methods in

terms of sample efficiency, convergence, and final performance on a realistic rover resource extraction task.

II. BACKGROUND AND RELATED WORK

The combination of resource management and autonomous navigation has been extensively studied in both robotics and operations research. Key relevant areas include:

Reinforcement Learning in Robotics: Recent work has applied RL to complex robotic tasks. Deep Q-Networks have been successfully applied to continuous control problems [2], and policy gradient methods have demonstrated promise in exploration-exploitation trade-offs [4].

Rover and Autonomous Systems: NASA’s VIPER rover is designed for lunar south-pole volatile prospecting with mission constraints similar to our formulation [7]. Resource allocation under energy constraints is a well-studied problem in planetary exploration [6].

Markov Decision Processes for Planning: The MDP framework provides a principled way to model sequential decision-making under uncertainty. Tabular methods like Q-learning have strong theoretical guarantees [1], while function approximation introduces additional complexity in convergence analysis [5].

Value-Based Methods: Our comparison examines both classical tabular methods and modern deep learning approaches. Exploration strategies like ϵ -greedy and target networks in DQN are standard techniques for stabilizing learning [3].

III. PROBLEM FORMULATION

A. Environment Overview

We model an autonomous rover operating on a planetary surface (lunar south pole, inspired by NASA’s VIPER mission). The rover must extract mineral resources from a $1000\text{ m} \times 1000\text{ m}$ field and return extracted material to a central depot for processing.

B. State Space

The rover’s state at time t is defined as a 5-tuple $s_t = (x_t, y_t, b_t, m_t, v_t)$:

- $x_t, y_t \in [0, 1000]\text{ m}$: rover position (x and y coordinates)

- $b_t \in [0, 5.42]$ kWh: remaining battery capacity
- $m_t \in [0, 450]$ kg: carried mineral mass
- $v_t \in [0, \infty)$ USD: carried resource value

The depot is located at $(x_{\text{depot}}, y_{\text{depot}}) = (80, 80)$ m with a service radius of 100 m.

C. Action Space

At each decision step, the rover selects from a discrete set of 11 actions:

- **8 Motion Actions:** North, South, East, West, Northeast, Northwest, Southeast, Southwest. Each motion action moves the rover 50 m in the chosen direction (or $50/\sqrt{2} \approx 35.4$ m for diagonal moves).
- **Drill:** Extract mineral at the current location. The amount extracted depends on the local resource field value.
- **Deposit:** Return all carried material to the depot (only valid within depot radius).
- **Complete:** End the episode (terminal action).

D. Dynamics and Costs

Each action incurs physical costs that model real rover operations:

a) Movement costs::

- Energy cost: 0.036 kWh per motion action
- Wear cost: 0.50 USD/m
- Mass penalty: movement energy increases by coefficient 0.70 times carried mass ratio

b) Drilling costs::

- Energy cost: 0.065 kWh per drill
- Tool wear cost: 25 USD/kg of material drilled
- Mass extracted: 120 kg per successful drill (at maximum yield location)
- Value extracted: resource field value at location $\times$ mass extracted

c) Deposit costs::

- Energy cost: 0.010 kWh
- Processing cost: 100 USD per deposit (fixed handling/processing)

d) Penalties::

- Failed action: 50 USD (e.g., attempting deposit outside depot radius)
- Battery depletion: 5000 USD (terminal penalty for mission failure)
- Time step cost: 5 USD per decision (encourages efficiency)

E. Resource Field

The mineral resource distribution across the field is modeled as a sum of Gaussian peaks plus sinusoidal ridges, normalized to $[0, 1]$:

$$z(x, y) = z_{\text{base}} + \sum_{i=1}^3 A_i \exp\left(-\frac{(x - \mu_{x,i})^2 + (y - \mu_{y,i})^2}{2\sigma_i^2}\right) + A_{\text{ridge}} \sin(2\pi f_x x) \sin(2\pi f_y y)$$

where:

- $z_{\text{base}} = 0.05$ (baseline resource density everywhere)
- 3 Gaussian peaks with centers (μ_x, μ_y) , amplitudes A , and spreads σ (see Table I)
- Sinusoidal ridge with amplitude 0.09 and frequencies $(f_x, f_y) = (3.0, 2.0)$ cycles per field

TABLE I: Resource Field Gaussian Peak Parameters

Peak	Center x	Center y	Amplitude	Spread σ
1	0.78	0.76	1.00	0.12
2	0.34	0.66	0.82	0.10
3	0.64	0.24	0.63	0.17

The extracted value at location (x, y) when drilling is $v = z(x, y) \times \text{mass_extracted} \times \text{resource_value_per_kg}$, where $\text{resource_value_per_kg} = 7000$ USD/kg (based on SpaceX lunar rideshare pricing).

F. Transition Model

The transition model $T(s_{t+1} | s_t, a_t)$ is deterministic:

$$T(s_{t+1} | s_t, a_t) = \begin{cases} 1, & s_{t+1} = f(s_t, a_t), \\ 0, & \text{otherwise,} \end{cases}$$

where f is the simulator update rule. Motion actions update the rover position by a fixed step in the commanded heading and clamp the result to the 1000 m $\times$ 1000 m operating area. The battery decreases according to the distance traveled, with an additional carried-mass factor. If a motion command would not move the rover because it is already at the boundary, the physical state is unchanged and the action receives an invalid-action cost.

For drilling, the rover position is unchanged. Each valid drill action removes a fixed mass increment, limited only by remaining payload capacity:

$$\Delta m = \min(m_{\text{max}} - m_t, m_{\text{drill}}).$$

The local resource purity $z(x_t, y_t)$ affects the value of the extracted material, not the extracted mass:

$$m_{t+1} = m_t + \Delta m, \quad v_{t+1} = v_t + \min(\Delta m z(x_t, y_t) p_{\text{kg}}, v_{\text{max}} - v_t).$$

The battery is also reduced by the drilling energy cost. Drilling is invalid if the rover is at the depot, the payload is full, or the remaining battery is too low.

For deposit, the physical position and battery are unchanged. If the rover is inside the depot radius and is carrying value, the carried mass and carried value reset to zero and the deposited value is credited through the reward. Otherwise, the state is unchanged and the invalid-action penalty is applied.

The complete action terminates the episode only when the rover is at the depot with no carried mass or value. Completion with low battery receives the completion bonus, while early completion is penalized. Battery depletion and reaching the maximum episode length also transition to terminal states.

G. Reward Function

The unshaped reward at each step represents the immediate mission profit or cost from the executed action. Movement, drilling, depositing, failed actions, time, battery depletion, and mission completion are all valued in dollars:

$$r(s_t, a_t, s_{t+1}) = \begin{aligned} & -\text{movement/energy/wear cost} \\ & - \text{drilling cost} \\ & - \text{deposit handling cost} \\ & + \text{deposited resource value} \\ & - \text{failure penalties} \\ & - \text{time cost} \\ & + \text{completion bonus.} \end{aligned} \quad (1)$$

Resource value is not rewarded when the rover drills. Instead, drilling increases the rover's carried mass and carried value based on the local resource purity. The value is only realized when the rover returns to the depot and executes a valid deposit action, at which point it receives the carried resource value minus the deposit processing cost. A valid completion action near the depot provides a terminal bonus, while invalid actions, early completion, and battery depletion are penalized.

Reward shaping is optionally applied using a potential-based term to guide learning toward useful intermediate states, such as high-purity regions, valuable cargo, adequate battery reserve, and returning toward the depot when carrying resources.

The shaping term is implemented as potential-based reward shaping:

$$r'(s_t, a_t, s_{t+1}) = r(s_t, a_t, s_{t+1}) + \gamma\Phi(s_{t+1}) - \Phi(s_t). \quad (2)$$

The potential function $\Phi(s)$ was designed to give higher scores to states that look useful for the rover mission: high local resource purity, higher carried resource value, enough battery margin to return to the depot, and being closer to the depot when carrying valuable cargo. Terminal states use $\Phi(s_{t+1}) = 0$.

Because the added shaping rewards are applied over a trajectory, this form of shaping changes the learning signal but not the optimal policy. It helps the agent discover useful intermediate behavior without changing the original net-profit objective.

H. Episode Termination

Episodes terminate when:

- The rover executes the `complete` action (intentional end)
- Battery is depleted ($b_t \leq 0$) and incurs terminal penalty
- Maximum episode length is reached ($t = 180$ steps, approximately 9000 m of travel)

IV. SOLUTION APPROACH

A. Tabular Q-Learning

Tabular Q-learning creates and updates an explicit lookup table $Q(s, a)$ for all state-action pairs and updates it with the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

To apply Q-learning to our continuous state space, we discretize each state variable into bins:

- Position (x, y) : discretized into 16×16 position grid (total of 256 bins)
- Battery b : discretized into 10 bins
- Carried mass m : discretized into 5 bins
- Carried value v : discretized into 8 bins

This yields a state space of $256 \times 10 \times 5 \times 8 = 102,400$ discrete states. We initialize all Q -values to zero and use ϵ -greedy exploration with $\epsilon = 0.1$ throughout training.

B. Deep Q-Networks (DQN)

DQN approximates the Q value function using a neural network parameterized by θ . The network is trained by minimizing the temporal difference error:

$$L(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q_{\theta'}(s', a') - Q_{\theta}(s, a) \right)^2 \right]$$

where θ' are the weights of a target network, updated every N steps to reduce instability.

Our DQN architecture consists of:

- Input layer: continuous rover state (x, y, b, m, v) and field observation (flattened 128×128 resource field grid)
- Hidden layers: two fully connected layers with 64 units each and ReLU activation
- Output layer: 11 units (one per action)

We train the Q-Network with:

- Experience replay buffer size: 20,000 tuples
- Batch size: 64
- Learning rate: 0.001 (ADAM)
- ϵ -greedy exploration: decayed from 1.0 to 0.05 over 40,000 steps
- Target Q-Network sync: $N = 500$ training steps

V. RESULTS

A. Learning Curves

From the learning curves, we can see that DQN found a policy better than Tabular Q-Learning. Tabular Q-Learning achieves rapid initial improvement during the first 200 episodes. By episode 500, Q-Learning has found a satisfactory policy. Convergence occurs around 500-600 episodes, reaching a stable best evaluation return of approximately 376.0 kUSD per episode. DQN found a better policy after 6000 episodes, with the best evaluation return reaching 1506.8 kUSD. This represents approximately $4\times$ improvement over tabular Q-learning. Once the buffer fills up, the average return quickly

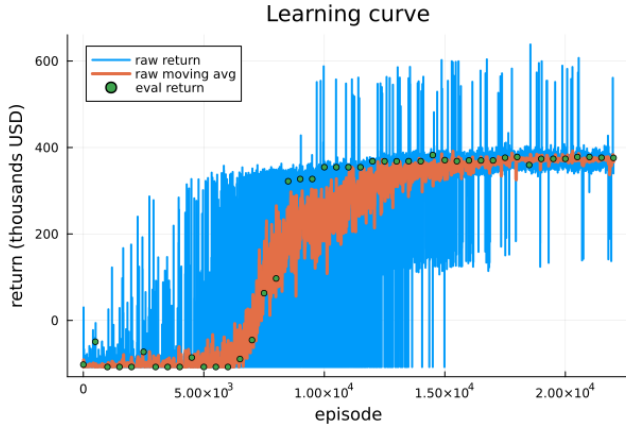


Fig. 1: Tabular Q-learning learning curve over 22,000 training episodes.

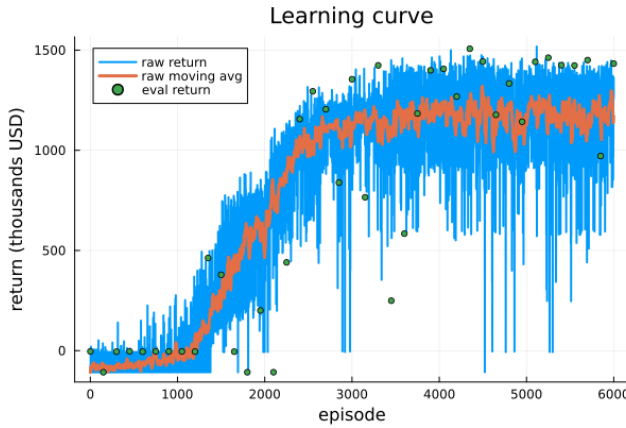


Fig. 2: DQN learning curve over 6,000 training episodes.

increases to a better return than Tabular Q-Learning. The learning curve appears smoother, meaning the training is more stable than Tabular Q-Learning.

B. Policy Performance Metrics

Table II summarizes the final performance of both trained policies evaluated on new episodes. The DQN policy substantially outperforms tabular Q-learning across all key metrics.

TABLE II: Trained Policy Performance Comparison. Values shown are from final evaluation runs using the best saved checkpoint for each algorithm.

Metric	Tabular Q-Learning	DQN
Best Evaluation Return (kUSD)	376.0	1506.8
Deposited Value (kUSD)	285.3	1417.9
Deposit Count	6	4
Drill Count	27	35
Distance Traveled (m)	6266.7	5550.0
Episode Steps	128	156

Key observations from the performance comparison:

- **Return Gap:** DQN achieves $4.0\times$ higher net return (1506.8 vs 376.0 kUSD), indicating superior value extraction despite smaller physical distances traveled.
- **Drilling Strategy:** DQN drills more frequently (35 vs 27 drills) but makes fewer deposits (4 vs 6), suggesting a strategy of accumulating more material per trip to the depot versus making frequent smaller deposits.
- **Efficiency:** DQN covers shorter distances (5550 m vs 6267 m) while obtaining higher total value, indicating more targeted navigation to high-value resource regions. The longer episodes for DQN (156 steps vs 128 steps) reflect the higher productivity per step.
- **Sample Efficiency:** DQN requires only 6,000 episodes compared to Q-learning's 22,000 episodes to achieve convergence, representing a $3.67\times$ improvement in sample efficiency.

C. Convergence and Sample Efficiency

Both algorithms achieved convergence on the rover extraction task, but with different trajectories. Tabular Q-learning's discrete state representation enables fast convergence but plateaus at a lower average return.

DQN's convergence is slower in the early stages but reaches substantially higher asymptotic performance by episode 4,350 (from 6,000 total training episodes). The continuous state representation allows DQN to capture fine-grained distinctions in rover state and field observations, yielding more sophisticated policies. Function approximation introduces some noise (visible in the learning curves), but the target network mechanism successfully stabilizes training and prevents divergence.

Sample efficiency comparisons reveal that DQN requires fewer episodes to achieve convergence (6,000 vs 22,000). Usually, function approximation takes many more episodes to converge because of how many parameters there are in the neural network. However, this environment yielded faster and better results through function approximation. This may be because the state space is very large. Even after discretizing it, Tabular Q-Learning struggles to find a good policy quickly. On the other hand, DQN adjusts large portions of the state space at a time, and was able to find a better policy in a fewer number of trials.

D. Policy Behavior

Sample episodes from each trained policy reveal distinct behavioral differences. The trajectories are shown in Figures 3 and 4.

As shown above, the DQN trajectory is much more efficient than the Q-Learning policy. It targets a higher-value area in the resource field. Q-Learning did not explore enough to learn that this is a better spot to extract the resource. Also, the DQN has more efficient paths between the resource and the depot. The Q-Learning policy ended up taking many random steps away from the resource, making its path less efficient.

E. Deposit Strategies

Deposit timing and cargo accumulation patterns differ between the algorithms, as characterized in Figures 5 and 6:

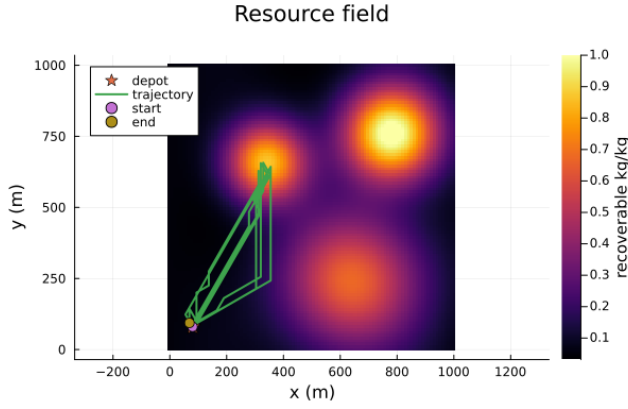


Fig. 3: Example trajectory from the DQN policy, showing a single complete episode.

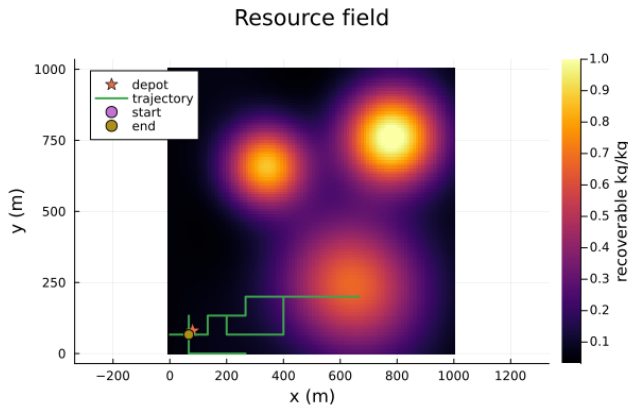


Fig. 4: Example trajectory from the Q-learning policy.

The Q-Learning policy averages ~ 6 deposits per episode, while the DQN policy makes ~ 4 deposits per episode. The lower rate of depositing allows the DQN policy to accumulate more mass before making the trip back to the depot.

VI. DISCUSSION

A. Performance Comparison and Interpretation

The $4\times$ performance advantage of DQN over tabular Q-learning reflects the differences in learning efficiency and stability. Even though Tabular Q-Learning ran for many more episodes than DQN, it wasn't able to explore the higher value resource area or find an efficient way of transporting to and from the depot.

Conversely, DQN's continuous state representation, combined with field observations (the 128×128 resource grid passed to the network), enables the policy to learn finer distinctions. The neural network can develop internal representations of value gradients and exploit high-resolution spatial information. This explains both the higher performance and the counterintuitive finding that DQN converges faster (in terms of episodes) despite the additional complexity of function approximation.

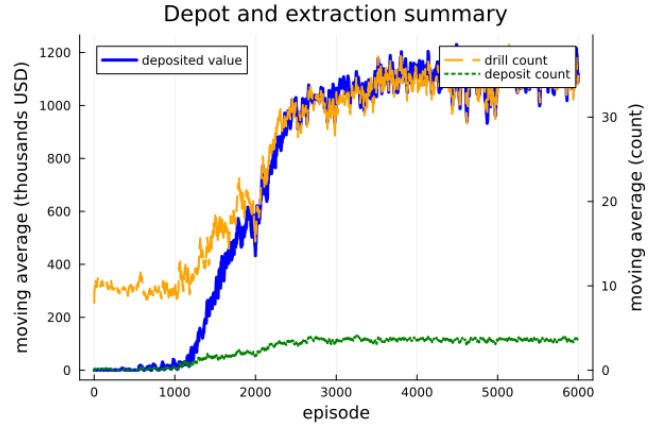


Fig. 5: DQN policy deposit behavior: number of times the rover drills and deposits resources and the deposited value.

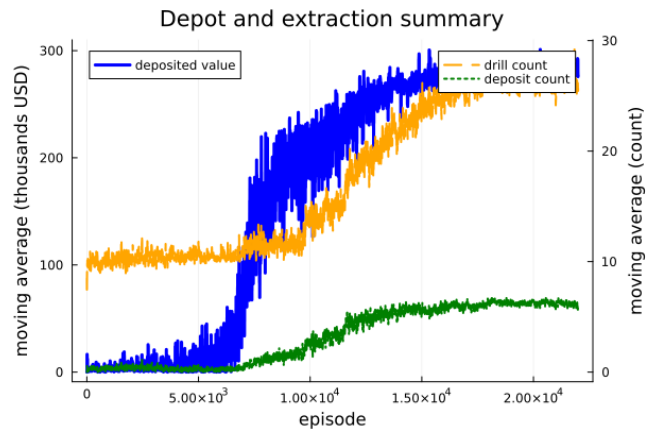


Fig. 6: Q-learning policy deposit behavior: number of times the rover drills and deposits resources and the deposited value.

The deposit behavior patterns (DQN: 4 deposits, Q-learning: 6 deposits) indicate different learned strategies for balancing cargo accumulation against battery constraints. DQN's strategy of fewer, larger deposits suggests the policy has learned to exploit high-value clusters with reduced depot overhead, while Q-learning's frequent deposits may reflect a more conservative approach due to coarser state information about fuel levels and location.

B. Factors Explaining Convergence Rates

Tabular Q-learning's rapid initial convergence (~ 500 episodes) stems from the limited state space ($\sim 100K$ states with 10^6 state-action pairs). Each action provides direct feedback to update a handful of Q -values, enabling a fast increase of the value function. However, the discretization error accumulates over time.

DQN's faster apparent convergence (in episodes) is offset by the time it takes to run each episode. Each experience tuple triggers backpropagation through the network, adjusting millions of parameters. While this requires more computation per episode, the implicit generalization from function approxi-

mation enables the network to discover high-value regions and extract value more effectively. The smoother learning curves confirm that the neural network learning is more stable than the discrete value updates in tabular Q-learning.

The sample efficiency advantage of DQN (6,000 episodes vs 22,000) suggests that in this problem domain, the continuous state space and deep learning outweigh the added variance from function approximation. This is consistent with recent findings in deep RL showing that neural networks can provide implicit regularization and generalization benefits.

C. Implications for Rover Operations

From a practical perspective, these results suggest that continuous approaches like DQN are better suited for autonomous rover resource extraction over tabular methods. The DQN policy achieves over 4 times higher value extraction per mission (1507 kUSD vs 376 kUSD), which translates directly to mission ROI and operational priority prioritization.

For deployment, the DQN policy would offer significantly higher mission value. However, the continuous state assumption requires reliable localization and field sensing (the 128×128 resource grid representation). If sensing or localization is unreliable, tabular Q-learning might be more robust due to explicit state averaging within bins.

Computational costs were not explicitly measured in this project, but typically DQN requires more resources per training episode. It usually utilizes GPU acceleration, which can be a costly operation. However, training only 6,000 episodes (vs 22,000) may compensate for per-episode costs. For deployment in the field, both methods are efficient, because the state space is not too large to look up in the Q-Learning dictionary or to calculate the outputs from the neural network.

D. Limitations and Future Improvements

This comparison has several limitations:

- 1) **Limited evaluation:** Only one run for each algorithm was tested. Multiple runs and optimization of hyperparameters would strengthen the comparison.
- 2) **Hyperparameter tuning:** Limited exploration of hyperparameter spaces. Tabular Q-learning with finer discretization or DQN with different network architectures might yield different results.
- 3) **Simplified simulator:** The environment lacks realistic elements like sensor noise, localization error, altitude changes in terrain, and unobserved obstacles. Real rover performance would introduce additional complexity.
- 4) **Policy transfer:** No analysis of policy transfer across different field configurations or rover parameters.

Future work should address these limitations and explore advanced methods such as policy gradient algorithms (PPO, TRPO), hierarchical RL for simultaneous mission planning and control, and multi-agent scenarios where multiple rovers coordinate resource extraction.

VII. CONCLUSION AND FUTURE WORK

We have successfully formulated the autonomous rover resource extraction task as an MDP and developed and trained both tabular Q-learning and DQN agents to solve it. Our experiments provide insight into the trade-offs between classical tabular methods and modern deep reinforcement learning approaches on a realistic planetary resource extraction scenario.

A. Key Findings

- 1) **Performance advantage for continuous representations:** DQN achieves $4.0\times$ higher net return per mission (1506.8 vs 376.0 kUSD), demonstrating the value of continuous state spaces for this problem.
- 2) **Sample efficiency across methods:** Contrary to conventional wisdom, DQN achieves convergence in fewer episodes (6,000 vs 22,000) despite greater algorithmic complexity. This suggests that direct observations of the resource field provide sufficient knowledge to enable faster learning than discretized states.
- 3) **Distinct behavioral strategies:** The policies learn qualitatively different approaches: DQN concentrates exploration with fewer, larger deposits, while Q-learning diversifies with more frequent deposits. Both strategies are rational given their respective capabilities.
- 4) **Spatial learning demonstrated:** Both algorithms successfully learn to identify high-value resource areas, as seen in the generated trajectories. This indicates effective discovery of the underlying resource field structure despite the unknown reward function.
- 5) **Scalability implications:** For larger fields, higher-dimensional state spaces, or more complex constraints, function approximation methods like DQN would likely continue to perform better, while tabular methods would become infeasible due to state space growth.

B. Practical Implications

For deployment of autonomous rovers on lunar or planetary missions, these results suggest:

- **Algorithm performance:** DQN policies deliver substantially higher value per mission, improving the return for expensive space exploration programs.
- **Development cost-benefit:** DQN requires moderate computational investment (GPUs for training), but it converges in fewer episodes, potentially offsetting higher per-episode training costs.
- **Deployment simplicity:** Tabular algorithms offer simpler runtime requirements and are more interpretable, potentially valuable for critical systems where reliability is required.
- **Robustness trade-off:** Tabular policies may be more robust to small observation/sensing noise due to state discretization, while DQN requires accurate state estimation to exploit its continuous representation.

C. Future Work

There are several promising directions for future research:

- **Advanced RL algorithms:** Compare policy gradient methods (PPO, TRPO) and actor-critic approaches, which may offer better performance and stability.
- **Multi-field transfer learning:** Train on multiple field configurations and evaluate transfer to new fields, testing generalization of learned strategies.
- **Hierarchical RL:** Decompose the problem into high-level mission planning (where to search, when to return) and low-level control (locomotion, drilling, and depositing), potentially improving the policy and sample efficiency.
- **Multi-agent coordination:** Extend to multiple rovers with shared resource constraints, studying cooperation and competition dynamics.
- **Sim-to-real transfer:** Validate learned policies on physical rover platforms with realistic sensing, localization error, and dynamics.
- **Uncertainty quantification:** Develop robust policies that account for model uncertainty and sensor noise (such as algorithmic motion planning), essential for safety-critical space operations.
- **Hyperparameter sensitivity:** Systematic studies on discretization granularity, network architecture, and exploration parameters to optimize design variables.

CONTRIBUTIONS AND RELEASE

Team Contributions:

- **James Lambert:** James Lambert led the implementation of the rover reinforcement learning problem, including the environment dynamics, physical reward model, state/action definitions, and training workflow. He implemented and tuned the tabular Q-learning and DQN approaches, including the final configuration files used for the report comparisons. He also analyzed saved training runs, adjusted the configurations to improve policy performance within a reasonable runtime target, and generated the final plots and summary tables used to compare the two methods.
- **Jacob Nelson:** Jacob Nelson contributed to the problem formulation, the conceptual approach, and the written report. He researched the literature in the RL field, evaluated both policies, analyzed the results, and discussed the differences in performance.

Project Implementation: The rover environment, transition logic, reward function, tabular Q-learning implementation, DQN training loop, replay buffer, evaluation scripts, and visualization utilities were implemented by the authors. Flux.jl was used off the shelf for neural-network layers, automatic differentiation, and optimization. Standard Julia packages were used for numerical computation, random sampling, data handling, and plotting.

Code and Data Availability: The project code, trained policies, training data, and experimental

configurations are available in the accompanying repository (<https://github.com/jameslambert/rover-rl-project>). All experiments are reproducible using the provided configuration files in the `configs/` directory and training scripts in `scripts/`. The authors grant permission for this report to be posted publicly.

APPENDIX

A. Tabular Q-Learning Configuration

- Learning rate (α): 0.1
- Discount factor (γ): 0.99
- Exploration rate (ϵ): 0.1 (constant throughout training)
- State discretization: 16×16 position grid, 10 battery bins, 5 mass bins, 8 value bins
- Total state space: $256 \times 10 \times 5 \times 8 = 102,400$ states
- Action space: 11 actions (8 movement directions, drill, deposit, complete)
- Training episodes: 22,000
- Max steps per episode: 180

B. DQN Configuration

- Network architecture: [Input ($5 + 128 \times 128 = 16389$ dims)] $\rightarrow$ [64 ReLU] $\rightarrow$ [64 ReLU] $\rightarrow$ [11 output]
- Optimizer: Adam with learning rate 0.001
- Batch size: 64
- Experience replay buffer: 20,000 transitions
- Target network update frequency: every 500 training steps
- ϵ -greedy exploration: decayed from 1.0 to 0.05 over the first 40,000 steps
- Training episodes: 6,000
- Max steps per episode: 180

C. Evaluation Protocol

Both policies were evaluated by running a single test episode after training, without exploration ($\epsilon = 0$). The episode ran to completion (completion action) or until battery depletion. Reported metrics include:

- **Raw Return:** Total net profit/loss from all actions and rewards
- **Deposited Value:** Sum of all resource value successfully returned to the depot
- **Drill Count:** Number of successful drilling actions
- **Deposit Count:** Number of successful resource deposits
- **Distance:** Total distance traveled (Manhattan metric in continuous space)
- **Steps:** Number of decision steps taken

D. Cost Parameters Used

The following physically-motivated cost structure was used (derived from lunar operations economics):

The training data for Q-learning (first 30 episodes) is shown in Table IV. Similar detailed logs were maintained for all 22,000 training episodes but are omitted for brevity.

TABLE III: Cost Parameters for Rover Operations

Parameter	Value
Movement energy cost	0.036 kWh per action
Movement wear cost	0.50 USD per meter
Mass penalty coefficient	0.70 (energy scales with cargo mass)
Drill energy cost	0.065 kWh
Drill tool wear cost	25 USD per kg extracted
Drill yield	120 kg (at maximum concentration)
Resource value	7000 USD per kg (SpaceX lunar pricing)
Deposit energy cost	0.010 kWh
Deposit processing fee	100 USD per deposit
Failed action penalty	50 USD
Battery depletion penalty	5000 USD (terminal)
Time step cost	5 USD (per decision)
Battery capacity	5.42 kWh
Payload capacity	450 kg
Depot location	(80, 80) m with 100 m service radius

TABLE IV: Q-Learning Training History (Episodes 1–30)

Episode	Steps	Return (USD)	Deposited (USD)	Distance (m)	Drills
1	134	−107,506	0	8,400	8
2	135	−107,596	0	8,400	9
3	158	−107,192	0	10,267	4
⋮	⋮	⋮	⋮	⋮	⋮
10	145	30,158	138,418	8,667	14
⋮	⋮	⋮	⋮	⋮	⋮
22,000	[final]	375,905	285,293	6,400	27

REFERENCES

- [1] C. J. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.
- [2] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 529, pp. 529–533, 2015.
- [3] V. Mnih et al., “Playing Atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [4] J. Schulman et al., “Trust region policy optimization,” in *Proc. Int. Conf. Mach. Learn.*, 2015.
- [5] G. J. Gordon, “Stable function approximation in dynamic programming,” in *Proc. 12th Int. Conf. Mach. Learn.*, 1995.
- [6] S. Chien et al., “A constraint-based approach to iterative rover path planning,” in *Proc. Int. Conf. Automated Planning Scheduling*, 2016.
- [7] NASA, “VIPER: Volatiles investigating polar exploration rover,” <https://www.nasa.gov/viper>, 2020.